

# PRINCIPLES OF DATABASE MODELING

## Complete Study Guide & SQL Administration Manual

A comprehensive synthesis of database architectures, schemas, relational modeling, normalization, access control, and partition scaling, fully grounded in the DataCamp course modules.

**Exam & Interview Fast-Track:** This study guide represents a systematic integration of theoretical slides, schema illustrations, SQL scripts, and administrative best practices. It has been compiled to serve as a definitive, single-source reference for relational database design, data warehousing paradigms, and PostgreSQL scaling strategies. All SQL examples utilize exact syntax validated in physical sandbox exercises.

### STRUCTURED SYSTEM INDEX

| Module & Focus Area                      | Key Analytical Competencies & Architectural Components                                                                                                                      |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Processing, Storing & Organizing Data | OLTP vs. OLAP architectures; Structured, semi-structured, and unstructured data; Traditional databases vs. Data Warehouses vs. Data Lakes; ETL vs. ELT ingestion pipelines. |
| 2. Database Design & Data Modeling       | Conceptual (ERD), Logical (Schemas), and Physical models; Relational mappings; High-level specifications for business databases.                                            |
| 3. Dimensional Modeling & Warehouses     | Star vs. Snowflake schema comparison; Fact and dimension tables; Querying mechanics and join differences; Schema extensibility.                                             |
| 4. Database Normalization & Integrity    | Core normalization rules; 1NF, 2NF, and 3NF definitions; Data anomalies (U-I-D); Relational integrity and composite primary keys.                                           |
| 5. Database Views & View Management      | Virtual tables and query-rewriting; Creating and querying views; Privileges (GRANT/REVOKE); Redefining views and updatability rules.                                        |
| 6. Materialized Views & Pipelines        | Physically materialized query results; Performance, freshness, and storage trade-offs; Dependency management and DAG scheduling.                                            |
| 7. Role-Based Access Control (RBAC)      | PostgreSQL roles, users, and groups; Least Privilege principles; Creating roles, granting privileges, and password attributes.                                              |
| 8. Physical Table Partitioning           | Vertical partitioning (separating wide columns); Horizontal list/range partitioning; Distributed sharding architectures.                                                    |
| 9. Data Integration Principles           | Combining heterogeneous data sources; Business-driven unified data models; Governance, lineage, security, and update cadences.                                              |
| 10. DBMS Selection & Decision Matrices   | SQL RDBMS vs. NoSQL systems; Key-value, document, columnar, and graph stores; Scenario-based decision matrices.                                                             |

# 1. PROCESSING, STORING, AND ORGANIZING DATA

## 1.1 OLTP vs. OLAP Processing Approaches

Relational database design is fundamentally governed by the intended business use-case of the data. Workloads are split into two distinct operational paradigms: Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP). OLTP supports immediate, high-volume transactional workloads, whereas OLAP supports deep historical aggregation, analytics, and business intelligence reporting.

| Feature               | OLTP (Online Transaction Processing)                                             | OLAP (Online Analytical Processing)                                                               |
|-----------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Support daily transactions and operational tasks                                 | Report and analyze consolidated, historical data                                                  |
| <b>Design</b>         | Application-oriented (operational snap-shots)                                    | Subject-oriented (historical analytical archives)                                                 |
| <b>Data State</b>     | Up-to-date, operational, normalized tables                                       | Consolidated, highly structured, historical, denormalized                                         |
| <b>Typical Size</b>   | Gigabytes (operational window)                                                   | Terabytes to Petabytes (long-term archive)                                                        |
| <b>Queries</b>        | Simple transactions, frequent insertions & updates                               | Complex, aggregate queries with limited updates                                                   |
| <b>Users</b>          | Thousands of concurrent active users (apps, clerks)                              | Hundreds of analytical consumers (analysts, executives)                                           |
| <b>Workload Type</b>  | Highly write-intensive (fast write priorities)                                   | Highly read-intensive (fast read priorities)                                                      |
| <b>Concrete Tasks</b> | Find price of a book; Update customer transaction; Keep track of employee hours. | Calculate books with best profit margin; Find most loyal customers; Decide employee of the month. |

**Exam Shortcut:** OLTP = operational + writes + current transactional snapshot; OLAP = analytical + reads + historical consolidated aggregates.

## 1.2 Data Types: Structured, Semi-structured, and Unstructured

Data ingested into modern systems spans multiple structural categories, each requiring different handling for storage and querying:

| Data Type              | Structural Description                                                                                                                             | Relational Compliance & Examples                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Structured</b>      | Follows a rigid, predefined schema. Highly organized into defined columns and rows with exact relationships.                                       | Highly compliant. Relational tables (SQL), structured columns (e.g., Integer, Varchar), primary/foreign keys.                   |
| <b>Semi-structured</b> | Does not follow a strict tabular schema, but possesses self-describing internal structures (like tags or key-value structures) that allow parsing. | Flexible. Represented by XML, JSON documents, or NoSQL databases. Attributes can be added dynamically without altering schemas. |
| <b>Unstructured</b>    | Schemaless. Lacks any predefined conceptual structure or organizational pattern. Comprises the vast majority of real-world data.                   | Completely schemaless. Stored as raw binaries (BLOBs), photos, chat logs, audio MP3s, videos, or raw text files.                |

**Example of Semi-structured JSON Schema:**

```
{
  "user": {
    "id": 23512,
    "profile_use_background_image": true,
    "statuses_count": 31,
    "profile_background_color": "C0DEED",
    "followers_count": 3066
  }
}
```

### 1.3 Traditional Databases vs. Data Warehouses vs. Data Lakes

Modern enterprise data architectures utilize three key storage paradigms to serve different functional requirements. While databases handle operations, warehouses and lakes consolidate data for analysis.

| Feature        | Traditional Database                      | Data Warehouse                                         | Data Lake                                                                |
|----------------|-------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------------|
| Data Structure | Real-time relational structured data      | Archived, consolidated structured data only            | Can include raw, structured, semi-structured, or unstructured data       |
| Core Purpose   | OLTP, real-time transactional operations  | OLAP, enterprise analysis, aggregation and reporting   | Big-data analytics, exploration, data discovery, deep learning           |
| Organization   | Schema-on-write, highly normalized design | Schema-on-write, denormalized, dimensional modeling    | Schema-on-read, highly flexible, raw, less organized structure           |
| Storage Cost   | High (premium operational media)          | Medium (optimized columnar database)                   | Low (bulk object storage, petabyte-scale scalable systems)               |
| Query Engine   | Relational row-store queries              | Massively Parallel Processing (MPP) analytical engines | Distributed big-data services (Apache Spark, Hadoop, Presto)             |
| Data Quality   | Highly organized and validated            | Highly transformed, clean, enterprise source of truth  | Raw data; requires careful cataloging to prevent becoming a 'data swamp' |

**Data Marts:** A Data Mart is a focused subset of a data warehouse dedicated to a specific business topic or functional area (e.g., Sales, HR, Marketing). It simplifies user access by isolating relevant dimensions and facts.

**Analytical Warehousing Technologies:** Modern warehouses like *Amazon Redshift*, *Azure SQL Data Warehouse*, and *Google BigQuery* utilize Massively Parallel Processing (MPP) specifically optimized for high-volume read operations.

### 1.4 Ingestion Pipelines: ETL vs. ELT

Ingestion processes load raw operational data into analytical environments. The sequence of transformation defines the architecture:

| Pipeline Process                  | Description of Transformation Flow                                                                                                                                                                                 | Typical Target Store                                                 |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| ETL<br>(Extract, Transform, Load) | Extract data from sources -> Transform (clean, standardize, mask) in a separate staging area -> Load into the warehouse. Transformations occur BEFORE data is written, ensuring only high-quality data enters.     | Structured Data Warehouse (e.g., for standard business reports)      |
| ELT<br>(Extract, Load, Transform) | Extract data from sources -> Load raw, unmodified data directly into the target -> Transform on-demand during query execution. Transformations are done INSIDE the system, leveraging scalable cluster processing. | Flexible Data Lake (e.g., for data science and big-data exploration) |

## 2. DATABASE DESIGN AND DATA MODELING

### 2.1 Foundations of Database Design & Schemas

Database design determines how data is logically stored, organized, and accessed. A key artifact of this design is the **schema**—the formal blueprint defining tables, fields, columns, relationships, constraints, indexes, and views. Relational systems strictly enforce these schemas, requiring all inserted data to conform to data types and constraint rules.

### 2.2 Three Levels of Data Modeling

Data modeling progresses from business concept down to database server hardware configurations. It is categorized into three discrete, sequential models:

| Model Level      | Primary Focus & Objective                                                                                                 | Industry Tools & Methods                                 | Mapping Elements                                       |
|------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------|
| Conceptual Model | Gathers business requirements. Establishes high-level business entities, and the core relationships between them.         | Entity-Relationship Diagrams (ERD), UML Diagrams         | Entities, attributes, high-level business rules.       |
| Logical Model    | Defines the database structure. Maps conceptual entities to concrete tables, attributes to columns, and establishes keys. | Relational Schemas, Star/Snowflake schemas, Normal forms | Tables, columns, data types, primary & foreign keys.   |
| Physical Model   | Focuses on disk storage and performance tuning. Describes the physical files, servers, indexes, and tablespaces.          | Table Partitioning, Indexing, backup systems, CPU tuning | Partitions, storage media, database engine parameters. |

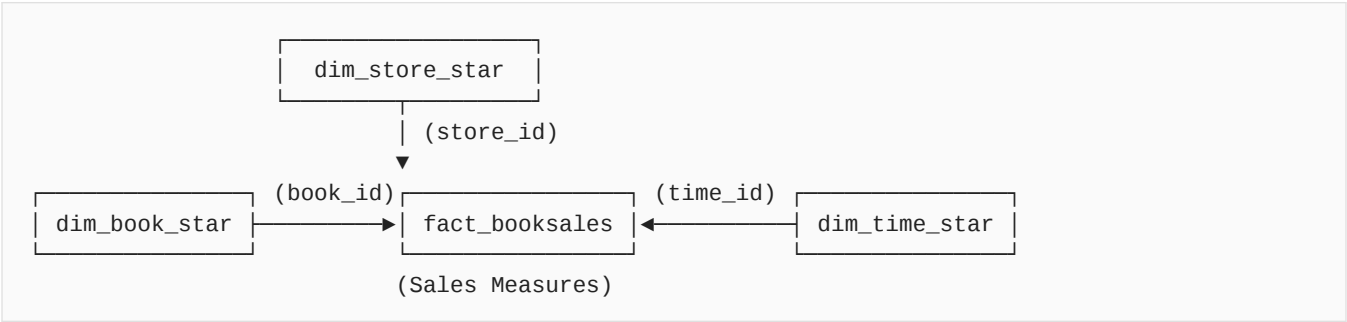
### 3. DIMENSIONAL MODELING & SCHEMAS

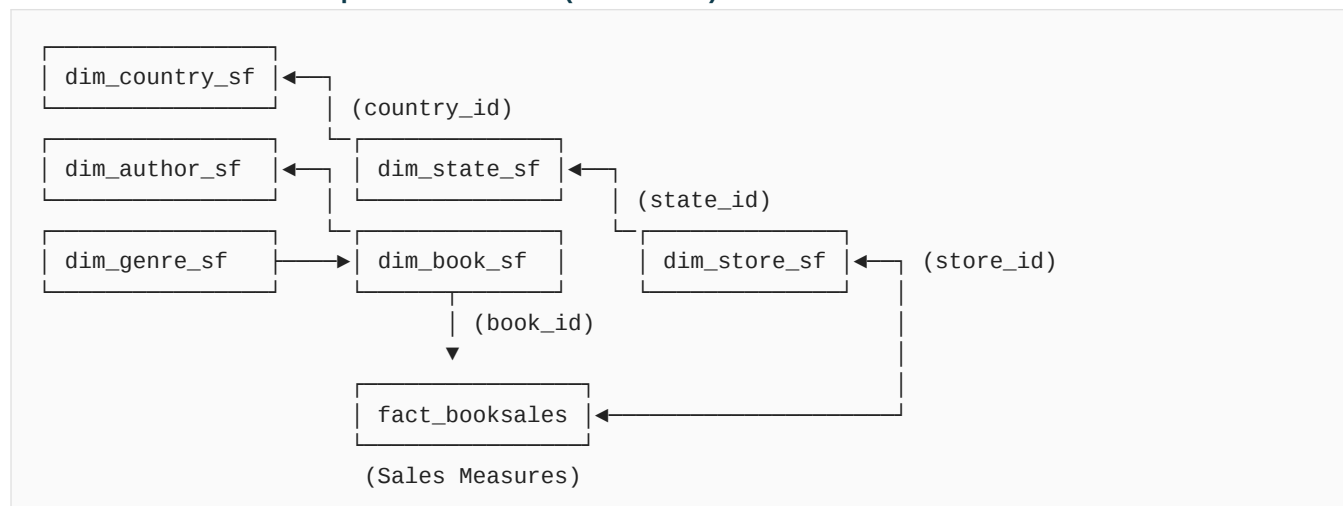
#### 3.1 Star Schema vs. Snowflake Schema Comparison

Dimensional modeling optimizes the relational database specifically for high-speed analytical aggregate queries (OLAP) by splitting tables into two functional types: Fact Tables (regularly updated records of key business metrics) and Dimension Tables (descriptive attributes of business entities that change infrequently).

| Feature            | Star Schema (Denormalized)                                                                                | Snowflake Schema (Normalized Dimension Tables)                                                                  |
|--------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Dimension Design   | Denormalized. Single table holds all related levels (e.g., City, State, and Country are in one table).    | Normalized. Dimension tables are split into multiple related tables (e.g., City -> State -> Country tables).    |
| Number of Tables   | Fewer tables (simpler schema layout)                                                                      | More tables (highly structured schema layout)                                                                   |
| Query Complexity   | Simpler queries. Fewer table joins are needed.                                                            | More complex queries. Multiple joins required to traverse relations.                                            |
| Read Performance   | Typically faster analytical reads (fewer JOINS means less CPU overhead).                                  | Can be slower on aggregates because multiple dimension tables must be joined.                                   |
| Data Consistency   | Lower. Denormalization allows data redundancy, risking inconsistent naming (e.g., 'California' vs. 'CA'). | Higher. Strict referential integrity forces naming consistency (e.g., centralizing countries in one table).     |
| Redundancy & Space | High data redundancy. Uses more physical storage space.                                                   | Minimal data redundancy. Saves physical disk space.                                                             |
| Extensibility      | Harder to extend. Adding attributes requires altering wide, highly replicated dimension tables.           | Easier to extend. Adding attributes (e.g., continent) only requires updating a small, specialized lookup table. |

Star Schema Conceptual Architecture:



**Snowflake Schema Conceptual Architecture (Normalized):**

### 3.2 Querying Star and Snowflake Schemas (SQL Comparison)

The difference in query complexity between star and snowflake schemas is demonstrated below. Both queries accomplish the exact same business goal: retrieve total sales of books in the 'novel' genre by state.

#### 1. Star Schema Query (Fewer JOINS, Simpler):

```
SELECT dim_store_star.state, SUM(fact_booksales.sales_amount)
FROM fact_booksales
JOIN dim_book_star ON fact_booksales.book_id = dim_book_star.book_id
JOIN dim_store_star ON fact_booksales.store_id = dim_store_star.store_id
WHERE dim_book_star.genre = 'novel'
GROUP BY dim_store_star.state;
```

#### 2. Snowflake Schema Query (More JOINS, Complex):

```
SELECT dim_state_sf.state, SUM(fact_booksales.sales_amount)
FROM fact_booksales
-- Joins to traverse the normalized book dimension to obtain the genre
JOIN dim_book_sf ON fact_booksales.book_id = dim_book_sf.book_id
JOIN dim_genre_sf ON dim_book_sf.genre_id = dim_genre_sf.genre_id
-- Joins to traverse the normalized store location dimensions to obtain the state
JOIN dim_store_sf ON fact_booksales.store_id = dim_store_sf.store_id
JOIN dim_city_sf ON dim_store_sf.city_id = dim_city_sf.city_id
JOIN dim_state_sf ON dim_city_sf.state_id = dim_state_sf.state_id
WHERE dim_genre_sf.genre = 'novel'
GROUP BY dim_state_sf.state;
```

### 3.3 Practice Exercise Case Study: Running Routine Dimensional Model

Imagine tracking a weekly running routine. The initial raw table runs contains duration\_mins, week, month, year, park\_name, city\_name, distance\_km, route\_name. To optimize for analytical reads, we model this as a star schema consisting of one central fact table and two dimension tables:

| Dimension Table             | Required Primary / Foreign Keys           | Descriptive / Metric Columns                  |
|-----------------------------|-------------------------------------------|-----------------------------------------------|
| <b>route_dim (or route)</b> | route_id (INTEGER PRIMARY KEY)            | route_name, park_name, distance_km, city_name |
| <b>week_dim (or week)</b>   | week_id (INTEGER PRIMARY KEY)             | week, month, year                             |
| <b>runs_fact</b>            | runs_id (PK), route_id (FK), week_id (FK) | duration_mins, distance_km                    |

**Important Typo Correction:** In some course exercise materials, the week dimension table was mistakenly written as route. Conceptually and structurally, it must be named week\_dim or week to avoid namespace collisions and enforce logical database clarity.

**SQL Creation Statements for Running Dimensions:**

```
-- Create the Route Dimension Table
CREATE TABLE route(
  route_id INTEGER PRIMARY KEY,
  route_name VARCHAR(160) NOT NULL,
  park_name VARCHAR(160) NOT NULL,
  distance_km INTEGER NOT NULL,
  city_name VARCHAR(160) NOT NULL
);

-- Create the Week Dimension Table (Typo corrected from 'route')
CREATE TABLE week(
  week_id INTEGER PRIMARY KEY,
  week INTEGER NOT NULL,
  month VARCHAR(160) NOT NULL,
  year INTEGER NOT NULL
);
```

### Querying the Running Star Schema:

```
-- Goal: Calculate the total running duration recorded in July 2019
SELECT SUM(runs_fact.duration_mins) AS total_duration_july
FROM runs_fact
INNER JOIN week_dim ON runs_fact.week_id = week_dim.week_id
WHERE week_dim.month = 'July' AND week_dim.year = 2019;
```

## 3.4 Dimensional Schema Operations & Extensions (SQL DDL)

To configure and maintain dimensional integrity or to transition layouts toward the snowflake model, DBAs execute the following DDL constraints and alterations:

### 1. Establishing Star Schema Foreign Key Integrity:

```
-- Apply foreign key constraints linking fact tables to central dimensions
ALTER TABLE fact_booksales ADD CONSTRAINT sales_book
    FOREIGN KEY (book_id) REFERENCES dim_book_star (book_id);

ALTER TABLE fact_booksales ADD CONSTRAINT sales_time
    FOREIGN KEY (time_id) REFERENCES dim_time_star (time_id);

ALTER TABLE fact_booksales ADD CONSTRAINT sales_store
    FOREIGN KEY (store_id) REFERENCES dim_store_star (store_id);
```

### 2. Star to Snowflake transition: Normalizing the Author Dimension:

```
-- Step A: Create the separate dimension table for authors
CREATE TABLE dim_author (
    author VARCHAR(256) NOT NULL
);

-- Step B: Ingest unique authors from the denormalized star book table
INSERT INTO dim_author(author)
SELECT DISTINCT author FROM dim_book_star;

-- Step C: Add an auto-incrementing surrogate primary key to complete the normalization
ALTER TABLE dim_author ADD COLUMN author_id SERIAL PRIMARY KEY;
```

### 3. Snowflake Schema Extension (Expanding Store to include Continent):

```
-- Step A: Alter the country table to add the foreign key with a default value of 1
ALTER TABLE dim_country_sf ADD COLUMN continent_id INT NOT NULL DEFAULT(1);

-- Step B: Link the new column to our lookup continent table via a foreign key
ALTER TABLE dim_country_sf ADD CONSTRAINT country_continent
    FOREIGN KEY (continent_id) REFERENCES dim_continent_sf(continent_id);
```

#### 4. Standardizing denormalized Star Schema country data:

```
-- Star schemas often contain country string inconsistencies due to human entries
-- Find all rows in dim_store_star that require updating to 'USA' or 'CA' abbreviations:
SELECT * FROM dim_store_star WHERE country != 'USA' AND country != 'CA';
```

## 4. DATABASE NORMALIZATION & INTEGRITY

### 4.1 Theoretical Principles of Normalization

Normalization is a database design methodology that systematically decomposes tables to eliminate data redundancy and protect database transaction integrity. By organizing attributes into their own, independent tables, we ensure that every data modification (INSERT, UPDATE, DELETE) only has to be written once, preventing database corruption.

### 4.2 Critical Operational Hazards: Data Anomalies

If databases are not sufficiently normalized, operations are vulnerable to three major anomalies, collectively remembered with the memory aid **U-I-D** (Update, Insertion, Deletion):

| Anomaly Type             | Description of Hazard                                                                                                                             | Real-World Student Enrollment Example                                                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Update Anomaly</b>    | Occurs when data redundancy requires updating identical attributes in multiple rows. If even one row is missed, data state inconsistencies occur. | To update student 520's email, the query must find and update multiple records. If some records are missed, the student will have different emails in the system.         |
| <b>Insertion Anomaly</b> | Occurs when we are unable to add a new record to the database because it requires other, currently missing, unrelated attributes.                 | We are unable to insert a newly signed-up student because the database requires them to be enrolled in a course (taught by an instructor) before a record can be written. |
| <b>Deletion Anomaly</b>  | Occurs when deleting a transactional record unintentionally results in the permanent loss of unrelated data attributes.                           | If we delete Student 230's enrollment, we unintentionally lose the only copy of information detailing the course 'Cleaning Data in R' and its instructor.                 |

### 4.3 The Design Dilemma: Benefits vs. Costs of Normalization

DBAs must strike a careful balance between normalization and performance depending on workload requirements:

| System Class                      | Normalization Level                         | Operational Advantages                                                                    | Design Drawbacks                                                                |
|-----------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>OLTP systems (Operational)</b> | Highly Normalized (3NF or higher)           | Eliminates redundancy, minimizes anomalies, ensures high-integrity fast write operations. | Requires numerous table joins, increasing CPU utilization and query complexity. |
| <b>OLAP systems (Analytical)</b>  | Less Normalized (Denormalized star schemas) | Prioritizes fast analytical read aggregates and simpler queries with fewer joins.         | Introduces major data redundancy, taking up larger storage footprints.          |

## 4.4 Progressive Database Normal Forms

To normalize a database, schemas are progressively refactored through mathematical normal forms (ordered from least to most normalized):

| Normal Form                        | Structural Rule Criteria                                                                                                                                            | Key Architectural Goal                                                                |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>1NF</b><br>(First Normal Form)  | 1. Every row/record must be completely unique.<br>2. All cell attributes must be strictly atomic (no repeating groups, arrays, or composite multi-valued strings).  | Enforce uniqueness and atomicity.                                                     |
| <b>2NF</b><br>(Second Normal Form) | 1. Must fully satisfy 1NF.<br>2. No partial dependency: If a composite primary key exists, non-key columns must depend on the ENTIRE primary key.                   | Eliminate partial dependencies.<br>(Memory Phrase: 2NF = no partial dependency)       |
| <b>3NF</b><br>(Third Normal Form)  | 1. Must fully satisfy 2NF.<br>2. No transitive dependency: Non-key columns cannot depend on other non-key columns (Attributes must depend only on the primary key). | Eliminate transitive dependencies.<br>(Memory Phrase: 3NF = no transitive dependency) |

## 4.5 Case Study: Step-by-Step SQL Normalization

This case study demonstrates the process of taking a car rental database from an unnormalized state through 1NF, 2NF, and 3NF using real SQL refactoring commands.

### 1. Converting to First Normal Form (1NF):

**Initial State:** The table customers contains single columns cars\_rented and invoice\_id holding multiple comma-separated values, violating atomicity.

**Refactoring:** Create a dedicated junction table to store individual rows, then drop the composite fields.

```
-- Create a separate junction table to hold atomic rentals
CREATE TABLE cust_rentals (
  customer_id INT NOT NULL,
  car_id VARCHAR(128) NULL,
  invoice_id VARCHAR(128) NULL
);

-- Remove the non-atomic multi-value columns from customers
ALTER TABLE customers
  DROP COLUMN cars_rented,
  DROP COLUMN Invoice_id;
```

### 2. Converting to Second Normal Form (2NF):

**Initial State:** The composite-keyed table customer\_rentals contains non-key columns model, manufacturer, type\_car, condition, color which depend only on car\_id, violating 2NF (partial dependency on part of a composite key).

**Refactoring:** Create a dedicated cars table and drop partial dependencies from the rentals table.

```
-- Create a dedicated cars lookup table to hold car attributes separately
CREATE TABLE cars (
  car_id VARCHAR(256) PRIMARY KEY,
  model VARCHAR(128),
  manufacturer VARCHAR(128),
  type_car VARCHAR(128),
  condition VARCHAR(128),
  color VARCHAR(128)
);

-- Populate the lookup table with distinct records
INSERT INTO cars
SELECT DISTINCT car_id, model, manufacturer, type_car, condition, color
FROM customer_rentals;

-- Remove the partially dependent columns from our transactional table
ALTER TABLE customer_rentals
  DROP COLUMN model, DROP COLUMN manufacturer,
  DROP COLUMN type_car, DROP COLUMN condition, DROP COLUMN color;
```

### 3. Converting to Third Normal Form (3NF):

**Initial State:** In our new cars table, the columns manufacturer and type\_car depend on model (non-key), which in turn depends on car\_id, creating a transitive dependency.

**Refactoring:** Create a dedicated car\_model table and drop the transitive attributes.

```
-- Create a separate model table to break transitive dependencies
CREATE TABLE car_model (
    model VARCHAR(128) PRIMARY KEY,
    manufacturer VARCHAR(128),
    type_car VARCHAR(128)
);

-- Drop the transitive fields from the primary transactional table
ALTER TABLE rental_cars
    DROP COLUMN manufacturer,
    DROP COLUMN type_car;
```

## 5. DATABASE VIEWS (VIRTUAL TABLES)

### 5.1 Database Views Architecture

A database View is a virtual table defined by a saved query. Views do not store data physically on disk (which distinguishes them from base tables). Instead, whenever a view is queried, the database engine retrieves and rewrites the underlying query on-the-fly, returning current transactional data from base tables.

| Key View Benefit        | Technical Operational Explanation                                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Zero Storage Overhead   | Views only store the query definition in memory, requiring no physical table storage on disk.                                    |
| Database Access Control | DBAs can restrict direct access to base tables while granting select access on views that hide sensitive columns.                |
| Mask Query Complexity   | Hides complex, nested database logic (multi-table joins, filters) behind a simple, easily queryable interface.                   |
| Schema Abstraction      | Allows DBAs to change underlying physical table schemas without breaking applications, by simply rewriting the view definitions. |

### 5.2 View DDL & Syntax (PostgreSQL Examples)

To define a standard view, DBAs execute a CREATE VIEW query. Once created, standard views can be queried just like physical tables.

```
-- Create a standard view to display science fiction books
CREATE VIEW scifi_books AS
SELECT title, author, genre
FROM dim_book_sf
JOIN dim_genre_sf ON dim_genre_sf.genre_id = dim_book_sf.genre_id
JOIN dim_author_sf ON dim_author_sf.author_id = dim_book_sf.author_id
WHERE dim_genre_sf.genre = 'science fiction';

-- Querying the virtual view
SELECT * FROM scifi_books;
```

### 5.3 Viewing Views in PostgreSQL (Information Schema)

To keep track of views in your database for documentation and lineage, you can query the system information schema, making sure to exclude built-in system views:

```
-- Retrieve all user-defined database views, excluding system catalogs
SELECT *
FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog', 'information_schema');
```

## 5.4 Advanced View Management and Security

Views are highly useful for access control and simplifying nested queries, allowing views to be built upon other views. However, DBAs must pay careful attention to privileges and updatability rules.

### 1. Creating a View from Other Views:

We can easily simplify queries by chaining views. For example, using two views (`top_15_2017` and `artist_title`) from the Pitchfork music reviews data to create a third view, `top_artists_2017`:

```
-- Chain views to create top_artists_2017
CREATE VIEW top_artists_2017 AS
SELECT r.artist
FROM artist_title AS r
INNER JOIN top_15_2017 AS t ON r.reviewid = t.reviewid;

-- Query the nested view
SELECT * FROM top_artists_2017;
```

### 2. Role-Based View Access Control:

To enforce security, access to the underlying tables can be completely blocked while specific privileges (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) are granted on views. This isolates transactional write capabilities:

```
-- Revoke update and insert privileges on views from general public
REVOKE UPDATE, INSERT ON long_reviews FROM PUBLIC;
REVOKE INSERT ON films FROM db_user;

-- Grant specific transactional write privileges to specialized roles
GRANT UPDATE, INSERT ON long_reviews TO editor;
GRANT UPDATE ON ratings TO PUBLIC;
```

### 3. Updatable Views and Base Table Modifications:

Not all views are updatable. Simple views derived from a single base table without aggregations, grouping, conditionals, or window functions allow inserts and updates to pass through to the base tables. To inspect which views in your schema support direct base table updates, execute the following system query:

```
-- Retrieve a list of views in your schema that support updates
SELECT table_name
FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
AND is_updatable = 'YES';
```

### 4. Redefining Views & Column Compatibility Rules:

To alter a view's underlying query, you can use `CREATE OR REPLACE VIEW`. This is subject to strict rules: the replacement query must preserve existing column names, data types, and order, and can only **append** new columns at the end of the query. If columns must be removed or renamed, you must drop and recreate the view.

```
-- Redefining a view by appending a column at the end
CREATE OR REPLACE VIEW artist_title AS
SELECT reviews.reviewid, reviews.title, artists.artist, labels.label
FROM reviews
INNER JOIN artists ON artists.reviewid = reviews.reviewid
INNER JOIN labels ON labels.reviewid = reviews.reviewid;
```

### 5. Dropping Views with Cascades:

If objects depend on a view, dropping that view will throw an error unless explicitly instructed. Use CASCADE or RESTRICT to govern this behavior:

```
-- RESTRICT (Default): Returns an error if other views/objects depend on this view
DROP VIEW view_name RESTRICT;

-- CASCADE: Automatically drops this view and any views/objects that depend on it
DROP VIEW view_name CASCADE;
```

## 6. MATERIALIZED VIEWS & PIPELINES

### 6.1 Materialized vs. Non-Materialized Views Trade-Offs

While standard views are virtual and execute on-the-fly, Materialized Views physically store the results of their underlying queries on disk. This introduces massive read speed advantages but introduces staleness considerations and storage footprints.

| Performance Aspect    | Non-Materialized View                                                       | Materialized View (MV)                                                              |
|-----------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Physical Storage      | None. Only stores the SQL query definition.                                 | Stored on disk. Occupies physical database storage.                                 |
| Read Performance      | Can be slow. Executes the base query every time it is read.                 | Extremely fast. Directly accesses the pre-computed physical table on disk.          |
| Data Freshness        | Always up-to-date. Automatically reflects real-time base table alterations. | Can become stale. Data remains a snapshot from the time it was last refreshed.      |
| Write-Heavy Databases | Better. Virtual queries add zero write overhead during transaction logs.    | Worse. Frequent refreshes can block database I/O.                                   |
| Optimal Use Case      | Standard operational databases with highly volatile data.                   | Expensive, long-running aggregate queries in read-intensive Data Warehouses (OLAP). |

### 6.2 Implementing and Refreshing Materialized Views (SQL Syntax)

To initialize a Materialized View, execute `CREATE MATERIALIZED VIEW`. To update its data to match base tables, you must explicitly call `REFRESH MATERIALIZED VIEW`:

```
-- Step A: Create a materialized view holding reviews counts by genre
CREATE MATERIALIZED VIEW genre_count AS
SELECT genre, COUNT(*)
FROM genres
GROUP BY genre;

-- Insert a new record into the base table
INSERT INTO genres VALUES (50000, 'classical');

-- Note: Querying genre_count now will NOT show 'classical' (stale data)
-- Step B: Refresh the materialized view to compute the classical entry
REFRESH MATERIALIZED VIEW genre_count;

-- Querying verified updated results
SELECT * FROM genre_count;
```

### 6.3 Managing Dependency Chains in Materialized Views

In large analytical warehouses, materialized views often depend on other materialized views. Refreshing views simultaneously is highly inefficient and can result in invalid/out-of-order data calculations.

**Dependency Chains & DAGs:** Database administrators map materialized view dependency chains using **Directed Acyclic Graphs (DAGs)**. DAGs guarantee that parent data is fully updated before any child view refresh is triggered.

**Scheduling tools:** Automated pipeline scheduling technologies like *Apache Airflow* or *Luigi* are deployed to orchestrate and execute refreshes in the exact mathematical order required by the DAG.

## 7. ROLE-BASED ACCESS CONTROL (RBAC)

### 7.1 Principles of Role-Based Access Control

Role-Based Access Control (RBAC) simplifies database security administration. Instead of granting permissions individually to each database user, DBAs assign privileges to logical database entities called **roles**. Users are then granted memberships in these roles, instantly inheriting all pre-configured permissions.

| RBAC Component                       | Strategic Advantage & Technical Objectives                                                                    |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Saves DBA Administration Time</b> | Instead of altering permissions for hundreds of users individually, permissions are managed in a single role. |
| <b>Persistence Across Users</b>      | Roles exist permanently as global server structures, living on even if employee user accounts are deleted.    |
| <b>Enforces Least Privilege</b>      | Restricts accidental excessive access by compartmentalizing specific privileges into precise task roles.      |

### 7.2 SQL Role Management Case Study (PostgreSQL DDL)

In PostgreSQL, both users and groups are represented as roles (users are simply roles with LOGIN enabled). This case study walks through setting up a startup database's security access schema.

```
-- Create a group role (by default cannot login)
CREATE ROLE data_scientist;

-- Create Marta's user role with LOGIN privileges
CREATE ROLE marta WITH LOGIN;

-- Create an administrative role with specific server rights
CREATE ROLE admin WITH CREATEDB CREATEROLE;

-- Grant specific privileges to the group role
GRANT UPDATE, INSERT ON long_reviews TO data_scientist;

-- Assign a secure login password to Marta's user role
ALTER ROLE marta WITH PASSWORD 's3cur3p@ssw0rd';

-- Enforce temporal group role membership (Grant Marta temporary data science access)
GRANT data_scientist TO marta;

-- Revoke membership once temporary tasks are complete
REVOKE data_scientist FROM marta;
```

### 7.3 Common PostgreSQL System Roles

PostgreSQL features pre-configured default system roles with precise administration capabilities:

| Default PostgreSQL Role     | Allowed Administrative Scope and Access Permissions                                                                                     |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>pg_read_all_settings</b> | Allows reading of all database configuration variables, including parameters normally visible only to superusers.                       |
| <b>pg_read_all_stats</b>    | Grants full read permissions on all system <code>pg_stat_*</code> views, and allows using various stats-related performance extensions. |
| <b>pg_signal_backend</b>    | Enables sending control signals to other background server connections (e.g., cancel long-running queries, terminate processes).        |

## 8. PHYSICAL TABLE PARTITIONING

### 8.1 Physical Performance Bottlenecks and Table Partitioning

As database tables grow to hundreds of gigabytes or terabytes, indexes become too massive to fit in physical server RAM. This causes query and update operations to slow down drastically due to heavy disk reading.

**Solution:** Partitioning physically splits a massive table into smaller, high-performance sub-tables (partitions).

Crucially, partitioning only affects the **physical data model**; the **logical data model** and client SQL statements remain completely unchanged.

| Partitioning Class      | Physical Implementation Concept                                                                                              | Key Performance Objective                                                                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Vertical Partitioning   | Splits wide tables by column. Specific columns are stored in separate, distinct physical files.                              | Moves infrequently accessed wide columns (like long descriptions) to slower/cheaper storage media, keeping hot attributes fast. |
| Horizontal Partitioning | Splits tables by rows based on a partitioning key (such as lists or date ranges).                                            | Ensures indexes of hot partitions (like the current quarter's sales) are small enough to fit entirely inside server memory.     |
| Sharding                | An extension of horizontal partitioning where horizontal row partitions are distributed across multiple independent servers. | Scales database resources out horizontally to achieve massive storage scaling and server fault tolerance.                       |

### 8.2 Implementing Vertical Table Partitioning

PostgreSQL does not feature a single native command for vertical partitioning. Instead, DBAs create a separate table to house wide columns, copy data there, drop columns from the primary table, and utilize JOIN clauses when full records are requested.

```
-- Step A: Create a separate vertical partition table for wide description files
CREATE TABLE film_descriptions (
    film_id INT,
    long_description TEXT
);

-- Step B: Ingest the data from the wide source table
INSERT INTO film_descriptions
SELECT film_id, long_description FROM film;

-- Step C: Drop the wide column from the hot primary table to optimize index sizes
ALTER TABLE film DROP COLUMN long_description;

-- Step D: Re-assemble the vertical partitions on-demand using a JOIN
SELECT * FROM film_descriptions JOIN film USING(film_id);
```

### 8.3 Implementing Native Horizontal List Partitioning (PostgreSQL DDL)

Native horizontal partitioning is configured by defining a parent table with a partitioning key (using PARTITION BY RANGE or PARTITION BY LIST) and registering sub-tables:

```
-- Step A: Create the partitioned parent table with LIST specifications
CREATE TABLE film_partitioned (
    film_id INT,
    title TEXT NOT NULL,
    release_year TEXT
) PARTITION BY LIST (release_year);

-- Step B: Define separate horizontal partition tables for distinct values
CREATE TABLE film_2019 PARTITION OF film_partitioned FOR VALUES IN ('2019');
CREATE TABLE film_2018 PARTITION OF film_partitioned FOR VALUES IN ('2018');
CREATE TABLE film_2017 PARTITION OF film_partitioned FOR VALUES IN ('2017');

-- Step C: Ingest rows into the parent table (rows route to sub-tables automatically)
INSERT INTO film_partitioned SELECT film_id, title, release_year FROM film;
```

## 9. DATA INTEGRATION ARCHITECTURE

### 9.1 Core Principles of Data Integration

Data Integration combines information from different sources, formats, and database systems (e.g., PostgreSQL, MongoDB, bulk CSV exports) to present users with a unified, translated, and business-driven view of organizational metrics (such as a 360-degree customer view or legacy systems consolidations).

| Integration Concern        | Technical Operational Best Practice Requirements                                                                                      |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Data Source Formats        | Enterprise integration frameworks must seamlessly ingest and parse heterogeneous schemas and file definitions.                        |
| Update Cadence             | Analytical views do NOT have to be updated in real-time by default. Update intervals should match business reporting requirements.    |
| Credit Card Anonymization  | PII and sensitive data (e.g., credit card primary account numbers) must be transformed and anonymized during the staging pipeline.    |
| Automated Testing & Alerts | Deploy automated testing suites and proactive alerts to quickly flag structural schema drift and pipeline failures.                   |
| Data Lineage & Governance  | Deploy visual lineage tracking to record and audit the exact origin and transformational steps of all enterprise metrics.             |
| Business-Driven Purpose    | Integration projects should be strictly business-driven, focusing only on useful combinations instead of integrating all raw records. |

### 9.2 High-Yield Data Integration True/False Decision Rules:

| Integration Statement                                              | Compliance Status | Architectural Rationale                                                                                                            |
|--------------------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Automated testing and proactive pipeline alerts are not needed.    | FALSE             | Testing and alerts are mandatory to prevent silent data pipeline failures and schema drift corruption.                             |
| All corporate data must be updated in real-time in the final view. | FALSE             | Most analytical metrics only require batch updates at scheduled intervals (e.g., daily or weekly) to align with business cadences. |
| Be extremely careful choosing a hand-coded integration solution.   | TRUE              | Custom hand-coded pipelines carry massive long-term code maintenance overheads compared to standardized integration frameworks.    |
| Data integration must always be strictly business-driven.          | TRUE              | Avoid integrating raw fields without a specific, high-value analytical objective.                                                  |

## 10. DBMS SELECTION & DECISION MATRICES

## 10.1 SQL vs. NoSQL Architectural Comparison

Selecting the right Database Management System (DBMS) depends entirely on the structure of your data and scalability requirements. RDBMS (SQL) systems prioritize strong consistency, whereas NoSQL systems prioritize schema flexibility and horizontal write scaling.

| Architectural Feature          | SQL / Relational Databases (RDBMS)                                                                   | NoSQL Databases                                                                                           |
|--------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Core Data Model</b>         | Tabular relational tables with strict predefined schemas.                                            | Flexible document, key-value pairs, columnar, or graph representations.                                   |
| <b>Schema Flexibility</b>      | Strict schema-on-write compliance is required.                                                       | Dynamic or schemaless on read (attributes can vary per row).                                              |
| <b>Transaction Consistency</b> | Strong consistency with strict ACID transaction guarantees.                                          | Eventual consistency models are typical (BASE properties).                                                |
| <b>Optimal Use Case</b>        | Structured data requiring strong consistency, referential integrity, and complex relational joining. | Rapid schema growth, highly flexible fields, massive unstructured data volumes, and horizontal scale out. |

## 10.2 The Four Core NoSQL Models & Use Cases

NoSQL engines are structurally grouped into four distinct models, optimized for specialized workloads:

| NoSQL Type      | Core Architectural Concept                                                                             | Enterprise Example Use Case                               | Industry Engines      |
|-----------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------|
| Key-Value Store | Associates a unique key string with an arbitrary data value. Extremely fast lookup operations.         | Managing buyer shopping carts during online sessions.     | Redis, Memcached      |
| Document Store  | Extends key-value stores by saving values as structured documents (typically JSON or BSON).            | Enterprise content management and user profiles.          | MongoDB, CouchDB      |
| Columnar Store  | Stores data grouped physically by columns instead of rows, drastically accelerating column aggregates. | Big-data analytical aggregates and logging repositories.  | Cassandra, HBase      |
| Graph Database  | Models data using nodes (entities) and edges (interconnected relationships).                           | Social media network analysis and recommendation engines. | Neo4j, Amazon Neptune |

## 10.3 Exam Scenario-Based DBMS Decision Matrix

This decision matrix isolates typical exam scenarios and the recommended database technologies:

| Scenario Description                                                           | Recommended Technology Class | Key Architectural Rationale                                                                             |
|--------------------------------------------------------------------------------|------------------------------|---------------------------------------------------------------------------------------------------------|
| Banking application requiring absolute data integrity and ACID safety          | SQL / RDBMS                  | Requires strong consistency, strict referential schemas, and zero transaction failure tolerances.       |
| A web blog supporting dynamic, varied content types (text, images, video tags) | NoSQL Document Store         | Requires highly flexible, dynamic, and unaligned schemas to ingest diverse attributes on-the-fly.       |
| A social media network mapping user profiles and their connections             | NoSQL Graph Database         | Graph nodes and edges natively navigate complex relationships without millions of expensive self-joins. |
| Enterprise data warehousing aggregates running on massive big-data volumes     | NoSQL Columnar Store         | Columnar stores physically group columns, allowing rapid reading of specific metrics across petabytes.  |
| E-commerce web portal tracking millions of simultaneous active buyer carts     | NoSQL Key-Value Store        | Requires ultra-low latency, simple key-based lookups to handle highly volatile session logs.            |

## 10.4 One-Page Memory Map of Database Design Concepts

This memory map consolidates the core concepts of the entire curriculum for high-speed study review:

| Analytical Category | Consolidated Core Concepts and Architectural Notes                                                                                               |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Workload Processing | OLTP: daily transactional, write-intensive, current operational snapshot.<br>OLAP: consolidated analytical, read-intensive, historical archives. |

| Analytical Category                | Consolidated Core Concepts and Architectural Notes                                                                                                                                                                                                                                                 |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Storage Infrastructure</b>      | Database (structured transactional) -> Data Warehouse (structured read-only, MPP) -> Data Marts (dedicated warehouse subsets) -> Data Lakes (unstructured raw data, Spark/Hadoop processing).                                                                                                      |
| <b>Data Modeling Level</b>         | Conceptual: Entity-Relationship Diagrams, UML, business requirements.<br/>Logical: Relational schemas, Star/Snowflake modeling, keys.<br/>Physical: Disk storage setups, indexing, CPU, tablespaces.                                                                                               |
| <b>Dimensional Modeling</b>        | Fact Table: holding metric measures, changes regularly.<br/>Dimension Table: descriptive attributes, changes infrequently.<br/>Star Schema: denormalized dimension tables directly connected to fact tables.<br/>Snowflake Schema: normalized dimension tables split into multiple related tables. |
| <b>Normalization Forms</b>         | 1NF: Unique rows, atomic values.<br/>2NF: 1NF compliant, no partial dependencies.<br/>3NF: 2NF compliant, no transitive dependencies.<br/>Anomalies: Update, Insertion, Deletion (U-I-D).                                                                                                          |
| <b>Database Views</b>              | Virtual Views: stored query definitions, zero storage overhead.<br/>Materialized Views: physically stored results, requires REFRESH, handles costly aggregates. Map dependencies using Directed Acyclic Graphs (DAGs) and pipeline schedulers (Airflow).                                           |
| <b>Security Administration</b>     | PostgreSQL Roles manage both users (LOGIN enabled) and group roles. Manage using GRANT and REVOKE following Least Privilege guidelines.                                                                                                                                                            |
| <b>Physical Table Partitioning</b> | Vertical: split table by columns (moves rare columns to slower media).<br/>Horizontal: split table by rows (list or range partitions, keeps index inside server memory).<br/>Sharding: distributes horizontal partitions across multiple machines.                                                 |

## 11. HIGH-YIELD SQL CHEAT SHEET & REFERENCE

This consolidated reference sheet serves as a quick-access cheat sheet for all DDL, DML, and DCL commands utilized across the course modules.

| SQL Operation                    | PostgreSQL DDL / DML / DCL Implementation Syntax                                                                                                    | Scope Objectives                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Create Base Table</b>         | <pre>CREATE TABLE table_name (   id INTEGER PRIMARY KEY,   name VARCHAR(256) NOT NULL );</pre>                                                      | Defines the physical table schema structure.                   |
| <b>Add Column &amp; Default</b>  | <pre>ALTER TABLE dim_country_sf ADD COLUMN continent_id INT NOT NULL DEFAULT(1);</pre>                                                              | Alters a table to append columns with a default value.         |
| <b>Add Foreign Key</b>           | <pre>ALTER TABLE child_table ADD CONSTRAINT fk_name FOREIGN KEY (child_id) REFERENCES parent_table (parent_id);</pre>                               | Enforces strict one-to-many referential integrity constraints. |
| <b>Create View</b>               | <pre>CREATE VIEW view_name AS SELECT col1, col2 FROM table_name WHERE condition;</pre>                                                              | Defines virtual views based on query definitions.              |
| <b>Redefine View</b>             | <pre>CREATE OR REPLACE VIEW view_name AS SELECT col1, col2, new_col FROM table_name;</pre>                                                          | Alters view definitions (must append columns to end).          |
| <b>Inspect Views</b>             | <pre>SELECT * FROM information_schema.views WHERE table_schema NOT IN ('pg_catalog', 'information_schema');</pre>                                   | Queries system catalogs to list all non-system views.          |
| <b>Inspect Updatable Views</b>   | <pre>SELECT table_name FROM information_schema.views WHERE table_schema NOT IN ('pg_catalog', 'information_schema') AND is_updatable = 'YES';</pre> | Identifies views that support base table updates.              |
| <b>Drop View Cascade</b>         | <pre>DROP VIEW view_name CASCADE;</pre>                                                                                                             | Removes a view and recursively drops all dependent objects.    |
| <b>Create Materialized View</b>  | <pre>CREATE MATERIALIZED VIEW mv_name AS SELECT * FROM existing_table;</pre>                                                                        | Physically saves query results to physical disk.               |
| <b>Refresh Materialized View</b> | <pre>REFRESH MATERIALIZED VIEW mv_name;</pre>                                                                                                       | Re-computes and syncs materialized views with base tables.     |
| <b>Create Login Role</b>         | <pre>CREATE ROLE marta WITH LOGIN;</pre>                                                                                                            | Creates a database user role allowed to login.                 |
| <b>Create Admin Role</b>         | <pre>CREATE ROLE admin WITH CREATEDB CREATEROLE;</pre>                                                                                              | Creates an administrative role with specific server rights.    |
| <b>Set Role Password</b>         | <pre>ALTER ROLE marta WITH PASSWORD 's3cur3p@ssw0rd';</pre>                                                                                         | Assigns/updates a secure authentication password.              |
| <b>Grant Privileges</b>          | <pre>GRANT SELECT, UPDATE, INSERT ON object_name TO role_name;</pre>                                                                                | Grants specific permissions on tables/views to roles.          |
| <b>Revoke Privileges</b>         | <pre>REVOKE UPDATE, INSERT ON object_name FROM PUBLIC;</pre>                                                                                        | Removes specific permissions on objects from roles.            |

| SQL Operation               | PostgreSQL DDL / DML / DCL Implementation Syntax                                              | Scope Objectives                                            |
|-----------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Add User to Group           | GRANT group_role TO user_role;                                                                | Adds user to a group role, inheriting all group privileges. |
| Vertical Partition          | CREATE TABLE target AS SELECT id, col FROM original;<br>ALTER TABLE original DROP COLUMN col; | Splits tables by column to reduce primary file widths.      |
| Horizontal Partition Parent | CREATE TABLE parent (<br>id INT, release_year TEXT<br>) PARTITION BY LIST (release_year);     | Initializes list-partitioned parent tables.                 |
| Create Sub-Partition        | CREATE TABLE part_2019 PARTITION OF parent FOR VALUES IN ('2019');                            | Creates sub-tables tied to specific partition key lists.    |